Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2026

DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA DE GESTIÓN DE BIBLIOTECA MEDIANTE ESTRUCTURAS DE DATOS NO LINEALES

202504863 - Kevin Ánderson Montejo Martínez

Resumen

El proyecto consiste en desarrollar un simulador del servicio de autorización de Documentos Tributarios Electrónicos (DTE) de la Superintendencia de Administración Tributaria (SAT) de Guatemala. El sistema recibe solicitudes de autorización de facturas, valida los datos del emisor y del receptor, verifica el NIT, comprueba el cálculo del IVA del 12 % y el total de la factura, y genera un código único para cada documento aprobado. La aplicación está desarrollada en C# y utiliza programación orientada a objetos, el protocolo HTTP y archivos JSON para intercambiar y almacenar información. Está dividida en dos componentes: Sat.Web, que permite enviar archivos y visualizar resultados, y Sat.Api, que procesa las solicitudes, almacena los datos de los contribuyentes y las facturas, y genera reportes estadísticos. Además, permite consultar facturas autorizadas, revisar movimientos de IVA y obtener información por contribuyente y rango de fechas. El proyecto busca aplicar los conocimientos de programación en el desarrollo de un servicio organizado, funcional y capaz de gestionar información de manera persistente.

Palabras clave.

Autorización, Contribuyente, Validación, Persistencia, API.

Abstract

The project involves developing a simulator for the Electronic Tax Document (DTE) authorization service provided by the Superintendency of Tax Administration (SAT) of Guatemala. The system receives requests for invoice authorization, validates the data of both the issuer and the recipient, verifies the Tax Identification Number (NIT), checks the calculation of the 12% VAT and the total invoice amount, and generates a unique code for each approved document. The application is developed in C# and utilizes object-oriented programming, the HTTP protocol, and JSON files for the exchange and storage of information. It is divided into two components: Sat.Web, which allows for the submission of files and the viewing of results, and Sat.Api, which processes requests, stores taxpayer and invoice data, and generates statistical reports. Furthermore, it enables users to query authorized invoices, review VAT transactions, and retrieve information based on specific taxpayers and date ranges. The project aims to apply programming knowledge toward the development of an organized, functional service capable of managing information in a persistent manner.

Keywords: *Authorization, Taxpayer, Validation, Persistence, API.*

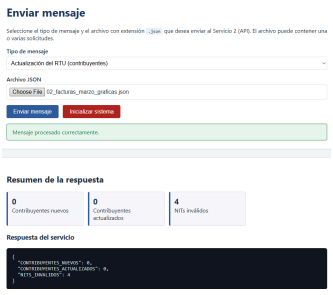
Introducción

El presente proyecto consiste en el desarrollo de un simulador del servicio de autorización de Documentos Tributarios Electrónicos (DTE) de la Superintendencia de Administración Tributaria (SAT) de Guatemala. Su propósito es representar el funcionamiento de un sistema que recibe solicitudes de autorización de facturas, verifica que cumplan con las reglas establecidas y genera un código único para identificar cada documento autorizado. De esta manera, se busca comprender cómo la tecnología puede contribuir a la gestión y validación de la información tributaria.

Para desarrollar esta solución se utiliza el lenguaje de programación C#, aplicando los principios de la programación orientada a objetos (POO), el protocolo HTTP y el intercambio de información mediante archivos JSON. El sistema se divide en dos componentes principales: una aplicación web que permite enviar solicitudes y visualizar resultados, y una API encargada de procesar los datos, validar la información de los contribuyentes y autorizar las facturas que cumplen con los requisitos establecidos. Además, la información se almacena de forma persistente para permitir consultas y generar reportes.

El desarrollo de este proyecto permite poner en práctica conocimientos fundamentales de programación, como la validación de datos, el diseño de algoritmos, el manejo de estructuras de datos y la comunicación entre aplicaciones. Asimismo, permite comprender la importancia de desarrollar servicios organizados, confiables y capaces de procesar información de manera consistente. El simulador constituye una oportunidad para aplicar los conocimientos adquiridos en el curso de Programación 2 mediante la construcción de una solución basada en una necesidad del contexto guatemalteco.

Imagen 1. Portada del proyecto



Fuente: Elaboración propia

Desarrollo del tema.

1. Objetivos y propósito del simulador SAT

El propósito principal del proyecto es desarrollar un servicio que simule el funcionamiento de un sistema de autorización de Documentos Tributarios Electrónicos (DTE) de la Superintendencia de Administración Tributaria (SAT) de Guatemala. Su función consiste en recibir solicitudes de autorización de facturas, verificar que cumplan con las reglas establecidas y generar un código único para identificar cada documento aprobado.

El sistema también permite registrar y actualizar información de los contribuyentes mediante el Registro Tributario Unificado (RTU), así como consultar datos y generar reportes relacionados con las facturas procesadas. De esta manera, el proyecto busca aplicar los conocimientos adquiridos en Programación 2 para construir una solución que procese información, valide datos y almacene resultados de forma organizada.

2. Programación orientada a objetos en C#

La programación orientada a objetos (POO) es uno de los fundamentos principales del proyecto, ya que permite organizar el software mediante clases y objetos que representan los elementos del sistema. A través del lenguaje C#, se pueden definir las propiedades y los métodos necesarios para administrar contribuyentes, facturas, solicitudes de autorización y reportes.

Este enfoque facilita la separación de responsabilidades y permite mantener el código más ordenado, comprensible y reutilizable. Además, el uso de estructuras de datos permite almacenar y manipular la información recibida. La aplicación de la POO contribuye a que el sistema sea más fácil de mantener, ampliar y modificar cuando sea necesario agregar nuevas funcionalidades.

3. Arquitectura del sistema: frontend y backend

La arquitectura del proyecto está dividida en dos componentes principales: el frontend y el backend. Esta separación permite distinguir la interfaz utilizada por

el usuario de los procesos internos encargados de validar y administrar la información.

El frontend, denominado Sat.Web, es una aplicación web que permite seleccionar archivos JSON, enviarlos al sistema y visualizar los resultados de las solicitudes. Por otra parte, el backend, denominado Sat.Api, recibe las peticiones mediante el protocolo HTTP, procesa la información, aplica las validaciones correspondientes y almacena los datos necesarios para las consultas y los reportes. Ambos componentes trabajan de manera coordinada para ofrecer una solución funcional y organizada.

Esta arquitectura también permite que la API sea consumida por otras herramientas, como Postman, sin depender exclusivamente de la interfaz web. Esto facilita las pruebas y la comprobación del funcionamiento de los servicios implementados.

4. Comunicación mediante HTTP y archivos JSON

La comunicación entre los componentes del proyecto se realiza mediante el protocolo HTTP, que permite enviar solicitudes y recibir respuestas entre aplicaciones. La API dispone de diferentes servicios para procesar actualizaciones de contribuyentes, solicitudes de autorización de facturas y consultas de información.

Los mensajes utilizados para intercambiar datos tienen formato JSON, una estructura que permite representar información mediante pares de propiedades y valores. Por ejemplo, una solicitud de autorización puede incluir la fecha, la referencia de la factura, los NIT del emisor y del receptor, el valor, el IVA y el total. La API recibe estos datos y devuelve una respuesta que indica los resultados del procesamiento, incluyendo las autorizaciones aprobadas y los rechazos correspondientes.

El uso de HTTP y JSON facilita la comunicación entre sistemas diferentes y permite transmitir información de manera estructurada. Asimismo, el servicio debe verificar que los mensajes recibidos tengan un formato válido, ya que los archivos malformados o con estructuras incorrectas pueden provocar respuestas de error. Estos mecanismos son

fundamentales para garantizar que la API procese las solicitudes de manera controlada y consistente.

5. Validación de contribuyentes y facturas

La validación de datos es una de las funciones más importantes del simulador SAT, ya que permite comprobar que las facturas cumplan con los requisitos establecidos antes de ser autorizadas. El sistema verifica que los NIT del emisor y del receptor sean válidos y que ambos contribuyentes se encuentren registrados en el Registro Tributario Unificado (RTU). También comprueba que la referencia de cada factura sea única y que la información numérica sea correcta.

Entre las validaciones financieras se encuentra el cálculo del Impuesto al Valor Agregado (IVA), correspondiente al 12 % del valor de la factura, redondeado a dos decimales. Posteriormente, se verifica que el total corresponda a la suma del valor y el IVA. Si una factura presenta errores, el sistema puede rechazarla y registrar el motivo correspondiente. Estas comprobaciones ayudan a mantener la integridad de la información y a evitar que se autoricen documentos con datos incorrectos.

6. Generación de códigos de autorización

El sistema genera un código único para cada factura que cumple con las validaciones establecidas. Este código permite identificar cada documento autorizado y está compuesto por la fecha de autorización y un número correlativo de ocho dígitos. Su estructura es yyyymmdd########, donde los primeros ocho dígitos representan el año, el mes y el día, mientras que los últimos ocho corresponden al correlativo.

El contador comienza en uno cada día y aumenta con cada nueva autorización, evitando que los códigos se repitan durante esa misma fecha. Por ejemplo, una autorización realizada el 1 de marzo de 2025 podría recibir el código 20250301000000001. Para mantener el control de estos números, el sistema almacena los correlativos y permite recuperarlos cuando se reinicia la aplicación. Este mecanismo facilita la identificación y organización de las facturas aprobadas.

7. Persistencia de datos, consultas y generación de reportes

La persistencia de datos permite conservar la información procesada por el sistema, incluso después de cerrar la aplicación. Para implementar esta funcionalidad se utilizan archivos JSON donde se almacenan los contribuyentes registrados, las facturas procesadas y los correlativos de autorización. Esto permite recuperar los datos cuando el servicio vuelve a ejecutarse, sin necesidad de registrar nuevamente toda la información.

Además, el simulador dispone de servicios para consultar facturas emitidas o recibidas por un contribuyente dentro de un rango de fechas, obtener resúmenes del IVA y calcular los valores autorizados durante determinados períodos. También permite generar reportes sobre errores encontrados, facturas aprobadas y otros datos estadísticos. Estas funcionalidades facilitan el análisis de la información almacenada y permiten conocer los resultados del procesamiento de las solicitudes.

8. Importancia, desafíos y aprendizajes del proyecto

El desarrollo del simulador SAT permite aplicar de manera práctica los conocimientos adquiridos en el curso de Programación 2. Entre los principales aprendizajes se encuentran la implementación de la programación orientada a objetos, el diseño de algoritmos de validación, el manejo de estructuras de datos, la comunicación mediante HTTP y el almacenamiento persistente de información. Asimismo, el proyecto permite comprender cómo una API puede ofrecer servicios que otras aplicaciones consumen sin depender de una interfaz específica.

Uno de los principales desafíos consiste en garantizar que las validaciones se ejecuten correctamente, especialmente al comprobar los NIT, calcular el IVA, detectar referencias duplicadas y generar códigos únicos. También es importante manejar adecuadamente los errores en los mensajes recibidos y mantener la información organizada para producir reportes confiables. En conjunto, el proyecto fortalece las habilidades de análisis, diseño y desarrollo de software, demostrando la importancia de construir sistemas estructurados, verificables y

capaces de administrar información de manera consistente.

Conclusiones.

1. El desarrollo del simulador SAT permitió aplicar los conocimientos adquiridos en el curso de Programación 2 mediante el uso del lenguaje C#, la programación orientada a objetos y la implementación de una API. Esto demuestra la importancia de organizar el código y separar las responsabilidades para construir aplicaciones funcionales y fáciles de mantener.

2. La implementación de validaciones, la generación de códigos de autorización y el almacenamiento persistente de información son elementos fundamentales para garantizar el correcto funcionamiento del sistema. El uso de HTTP y JSON facilita la comunicación entre aplicaciones, mientras que los archivos JSON permiten conservar los datos de contribuyentes, facturas y correlativos para realizar consultas y generar reportes.

3. El proyecto permitió comprender cómo los conocimientos de programación pueden utilizarse para desarrollar soluciones relacionadas con procesos tributarios. Asimismo, contribuyó al fortalecimiento de habilidades de análisis, diseño de algoritmos y manejo de datos, necesarias para desarrollar sistemas confiables que procesen información de manera organizada y consistente.

Bibliografía:

- *guardrex. (2026, October 10). Tutorial: Creación de una API web basada en controlador con ASP.NET Core. MicrosoftLearn. https://learn.microsoft.com/es-es/aspnet/core/tutorials/first-web-api?view=aspnetcore-10.0*
- Amazon Web Services. (2026, October 16). ¿Qué es una interfaz de programación de aplicaciones (API)? Amazon Web Services, Inc. https://aws.amazon.com/es/what-is/api/
- Desarrollo de API: Qué lenguaje de programación usar. (2024, May 2). Rootstack. https://rootstack.com/es/blog/desarrollo-de-api-lenguaje-de-programacion

DIAGRAMAS

